LEC-05

1. TL;DR / Executive Summary

- **Advanced Context Control:** Skills can be isolated to sub-agents via metadata instructions, keeping the main context window clean while leveraging custom models or manual invocation locks.
- **Session Lifecycle & Forking:** Complex workflows are managed using session renaming (/rename), session branching (/copy), history compaction for token management, and session resumption (/resume).
- **External Integration via MCP:** Model Context Protocol (MCP) servers enable robust external tool connectivity through mcp.json configurations, authenticated verification (/mcp), and local skill directories (.claudes/skills/).
- **Workflow Modes & Troubleshooting:** Operational modes (Shift + Tab) like Plan Mode and Interview Mode structure interaction, while environment consistency is maintained by auditing auto-generated memory.md files.

2. Core Concepts & Definitions

Term / Concept	Definition / Explanation
Sub-Agent Skill Isolation	Assigning skills to sub-agents via context: sub-agent meta instructions to run tasks outside the primary context window.
Session Forking (/copy)	Creating an exact branch/copy of an active session to test experimental workflows without disrupting the original state.
Compaction	An automated mechanism that summarizes oversized conversation histories to fit within performance and token limits.
Plan Mode	An operational mode forcing Claude to draft a detailed execution plan for manual review before making any code or file modifications.
Interview Mode	A workflow technique instructing Claude to proactively ask clarifying questions before generating content to ensure precision.
Auto Memory (memory.md)	A persistence file where Claude automatically logs project learnings, which can be modified or disabled to maintain deterministic outputs.

3. Structured Body Hierarchy

- **Advanced Skill Management & Context Isolation**
  - **Sub-Agent Delegation:** Use context: sub-agent in skill.md headers to isolate procedural overhead from the main session.
  - **Execution Controls:** Restrict or customize execution by assigning specific models (e.g., Sonnet) or disabling automatic invocation so skills only run on direct user command.
- **Session Management and Persistence**
  - **State Organization:** Rename active environments using /rename and fork workflows via /copy for safe experimentation.
  - **Memory & Continuity:** Leverage automated history compaction when approaching token limits, and reload past environments using /resume.
- **Integrating External Tools with MCP**
  - **Configuration:** Define server definitions and endpoints inside global or project-level mcp.json files.
  - **Verification & Placement:** Authenticate connections via browser prompts, use /mcp to verify tools, and store custom project skills in .claudes/skills/.
- **Advanced Workflow Modes**
  - **Operational Modes (Shift + Tab):** Toggle between Plan Mode (for pre-execution review), Manual/Auto modes, and Interview Mode (for requirement alignment).
- **Troubleshooting Environment Inconsistencies**
  - **Determinism Control:** Investigate auto-generated memory.md files when cross-machine outputs diverge due to accumulated project learnings.

4. Review & Active Recall (Q&A)

- **Q: How does assigning a skill to a sub-agent improve performance in Claude Code?**
  - **A:** It isolates the skill's execution from the main conversation, preventing token bloat and keeping the primary context window clean and focused.
- **Q: What is the difference between /copy and /resume in session management?**
  - **A:** /copy forks an active session to test new ideas safely without altering the original, whereas /resume restores access to a previously saved session.
- **Q: What is the purpose of Plan Mode before executing file changes?**
  - **A:** It forces Claude to outline a detailed, step-by-step implementation strategy for human review prior to running tool calls or modifying code.
- **Q: Why might two identical project setups yield different outputs across machines, and how is it fixed?**
  - **A:** Differences arise from Claude's Auto Memory feature writing distinct project learnings into memory.md, which can be adjusted or disabled to ensure determinism.